LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack & Queue

Oleh:

Laily Azizah Islami NIM. 2510817220020

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL

2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Laily Azizah Islami
NIM : 2510817220020

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1: Implementasi Stack 1
A. Source Code 1
B. Pembahasan 3
SOAL 2: Si Palui dan Mesin Hitung Aneh 5
A. Source Code 6
B. Output Program 8
C. Pembahasan 8
SOAL 3: Implementasi Queue 11
A. Source Code 11
B. Pembahasan 13
Soal 4: Si Palui dan Deretan Angka 16
A. Source Code 17
B. Output Program 18
C. Pembahasan 18
TAUTAN GIT 20

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2A 8
Gambar 2. Screenshot Hasil Jawaban Soal 2B 8
Gambar 3. Screenshot Hasil Jawaban Soal 2C 8
Gambar 4. Screenshot Hasil Jawaban Soal 4A 18
Gambar 5. Screenshot Hasil Jawaban Soal 4B 18
Gambar 6. Screenshot Hasil Jawaban Soal 4C 18

DAFTAR TABEL

Tabel 1. Source Code Stack.cpp 1
Tabel 2. Source Code prob1.cpp 6
Tabel 3. Source Code queue.cpp 11
Tabel 4. Source Code prob2.cpp 17

SOAL 1: Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation)

A. Source Code

Tabel 1. Source Code Stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::out_of_range("Stack terisi penuh");
    }
```

```

    if (isEmpty(s)) {
        s->top = &(s->data[0]);
    } else {
        s->top++;
    }

    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
         throw std::underflow_error("Stack kosong, tidak ada yang bisa diambil");
    }

    if (s->top == &(s->data[0])) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
         throw std::underflow_error("Stack kosong, Tidak ada data yang dapat dilihat.");
    }

    return *(s->top);
}
```

B. Pembahasan

1. Penjelasan Fungsi: Pada baris [1] `#include "stack.h"` yang digunakan untuk mengimpor definisi dari struktur `stack.h` agar dapat digunakan pada program ini. Pada baris [2] terdapat `#include <stdexcept>` yang digunakan untuk memanggil standar exception seperti `out_of_range` dan `underflow_error` yang akan digunakan dalam menangani kondisi tidak valid. Selanjutnya pada baris [4] hingga [6] terdapat fungsi `init` yang akan menginisialisasi pointer `top` ke `nullptr` yang berarti stack berada pada kondisi kosong karena tidak menunjuk ke alamat memori manapun. Selanjutnya pada baris [8] hingga [10] terdapat fungsi `isEmpty` yang akan memeriksa apakah pointer top masih bernilai `nullptr`. Jika pointer top masih bernilai `nullptr`, maka stack akan dinyatakan kosong. Pada baris [12] hingga [14] terdapat fungsi `isFull` yang akan memeriksa apakah alamat memori yang ditunjuk oleh `top` sudah sama dengan alamat yang ada pada elemen terakhir dalam array data. pada baris [13] terdapat `data[MAX - 1]` yang digunakan untuk batas akhir atau kapasitas maksimum dalam array data. Pada baris [16] terdapat deklarasi fungsi `void push(Stack* s, int value) {` yang akan digunakan untuk menambahkan data ke dalam stack, sehingga fungsi ini akan menerima parameter berupa pointer ke dalam stack. Pada baris [17] hingga [19] terdapat pengecekkan apakah nilai dari `isFull` telah penuh. Jika nilai `isFull` terpenuhi, maka program akan melempar exception `out_of_range` dan menampilkan kalimat stack terisi penuh. Selanjutnya baris [21] yang berfungsi sebagai pengkondisian kedua yang akan mengecek apakah ini adalah pengisian data pertama. Jika pengkondisian pada baris [21] terpenuhi maka baris [22] akan dieksekusi dengan mengarahkan pointer `top` ke alamat indeks ke-0 dari stack jika stack sebelumnya kosong. Pada baris [23] terdapat pengkondisian yang menjadi alternatif jika kondisi `if` pada baris sebelumnya tidak terpenuhi. Jika kondisi pada baris [23] terpenuhi maka program akan menaikkan alamat pointer ke posisi selanjutnya yang ada di dalam stack. Pada baris [27] terdapat `*(s->top) = value;` yang berfungsi untuk memasukkan nilai `value` ke dalam alamat memori yang saat ini sedang ditunjuk oleh `top`. Selanjutnya pada baris [30] hingga [33] terdapat fungsi `pop` yang akan akan melakukan pengecekkan fungsi `isEmpty`. Jika fungsi `isEmpty` bernilai benar, maka program akan melempar exception dan kalimat bahwa stack masih kosong. Selanjutnya pada baris [35] dan [36] terdapat pengkondisian kedua yang akan mengecek apakah data yang dihapus merupakan elemen terakhir yang berada pada stack. Setelah mengecek elemen terakhir yang diambil, baris [36] berkerja untuk mengembalikan status stack menjadi kosong. Pada baris [37] dan [38] terdapat pengekondisian terakhir yang akan menjadi alternatif jika elemen yang dihapus bukan elemen terakhir dari stack. Baris [38] yang berisi `s->top--;`

akan berfungsi untuk menurunkan alamat pointer ke elemen yang ada dibawahnya. Lalu pada baris [39] terdapat simbol kurung kurawal yang menjadi penutup dari pengkondisian `else`. Sedangkan pada baris [40] menjadi penutup dari fungsi `pop`. Selanjutnya pada baris [42] terdapat deklarasi fungsi `int peek(const Stack* s)` yang akan digunakan untuk melihat nilai yang berada paling atas dari sebuah stack tanpa harus menghapusnya. Pada baris [43] terdapat pengkondisian `if (isEmpty(s)) {` yang akan mengecek apakah data yang ingin dilihat tersedia. Jika kondisi pada baris [43] terpenuhi, maka baris [44] akan dijalankan. Baris ini akan menjalankan `throw std::underflow_error("Stack kosong, Tidak ada data yang dapat dilihat.");` yang akan menampilkan kalimat bahwa stack kosong, sehingga tidak ada data yang dapat dilihat. Lalu pada baris [45] terdapat simbol tutup kurung yang akan menutup pengkondisian `if`. Pada baris [47] terdapat `return *(s->top);` yang akan mengembalikan nilai dari alamat memori yang ditunjuk oleh pointer `top`. Terakhir, pada baris [48] ditutup oleh simbol kurung kurawal yang menutup blok fungsi `peek`.

2. Penjelasan Kebenaran Algoritma: Pada program stack ini, program menggunakan implementasi dari prinsip Last-In-First-Out atau LIFO. Hal ini didukung oleh penggunaan fungsi push yang akan menempatkan elemen baru diposisi teratas (top), lalu pada program ini terdapat fungsi pop yang akan mengambil elemen terakhir ynag dimasukkan. Penggunaan exception handling dengan throw akan mencegah kondisi yang mengambil data kosong (underflow) atau menambah data saat kapasitas stack dalam keadaan penuh (overflow), sehingga struktur dan proses kerja stack tetap terjaga.

3. Analisis Kompleksitas (Big-O Notation): Pada program stack ini terdapat beberapa fungsi, diantaranya push, pop, top, peek, isEmpty, dan isFull. Fungsi-fungsi tersebut memiliki kompleksitas waktu dan ruang sebesar $O(1)$ atau konstan karena fungsi-fungsi tersebut fijalankan dalam waktu tetap (konstan) tanpa melalui proses iterasi atau perulangan terhadap jumlah data. Fungsi-fungsi tersebut juga menggunakan beberapa variabel tambahan yang ukurannya tetap, diluar dari seberapa besar kapasitas stack yang digunakan.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

• sebuah bilangan bulat, atau

• sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

• Mesin membaca simbol dari kiri ke kanan.

• Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.

• Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n adalah jumlah simbol dalam ekspresi

• A_I adalah nilai bilangan pada input

• Simbol yang valid hanya:

- bilangan bulat
- operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang simbol

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 2. Source Code prob1.cpp

```
1  #include "stack.h"
2  #include <iostream>
3  #include <string>
4
```

```
int main() {
    int n;
    if (!(std::cin >> n)) return 0;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        std::string simbol;
        std::cin >> simbol;

         if (isdigit(simbol[0]) || (simbol.size() > 1 && simbol[0] == '-')) {
            push(&s, std::stoi(simbol));
        } else {

            int v2 = peek(&s); pop(&s);
            int v1 = peek(&s); pop(&s);

            if (simbol == "+") push(&s, v1 + v2);
            else if (simbol == "-") push(&s, v1 - v2);
            else if (simbol == "*") push(&s, v1 * v2);
            else if (simbol == "/") push(&s, v1 / v2);
        }
    }
    std::cout << peek(&s) << std::endl;
    return 0;
}
```

B. Output Program

```
PROBLEMS    DEBUG CONSOLE    PORTS    TERMINAL    powershell

PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2> cd stack-and-queue
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2\stack-and-queue> .\prob1
3
2 3 +
5
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2\stack-and-queue>
```

Gambar 1. Screenshot Hasil Jawaban Soal 2A

```
PROBLEMS    DEBUG CONSOLE    PORTS    TERMINAL    powershell

PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2> cd stack-and-queue
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2\stack-and-queue> .\prob1
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2\stack-and-queue>
```

Gambar 2. Screenshot Hasil Jawaban Soal 2B

```
PROBLEMS    DEBUG CONSOLE    PORTS    TERMINAL    powershell

PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2> cd stack-and-queue
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2\stack-and-queue> .\prob1
5
10 2 / 3 *
15
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK AL
GO SEM2\stack-and-queue>
```

Gambar 3. Screenshot Hasil Jawaban Soal 2C

C. Pembahasan

Pada baris [1] terdapat `#include "stack.h"` yang digunakan untuk mengimpor stack agar dapat digunakan pada program ini. Pada baris [2] terdapat `#include <iostream>` yang digunakan untuk mengimpor library `iosteram` agar program dapat menggunakan standar input dan output. Pada baris [3] `#include <string>` yang akan digunakan agar program dapat menggunakan tipe data string. Selanjutnya pada baris [5] terdapat `int main()` yang menjadi penanda awal untuk eksekusi program. Pada baris [6] terdapat `int n;` yang mendeklarasikan variabel `n` dengan tipe data integer.

Variabel ini akan digunakan untuk menyimpan berapa jumlah simbol dari input. Selanjutnya pada baris [7] terdapat pengkondisian `if (!(std::cin >> n)) return 0;` yang digunakan untuk membaca input dan menyimpannya pada variabel `n`. Pada baris [9] terdapat `stack.s` yang akan digunakan untuk menyimpan memori angka. Pada baris [10] `init(&s);` yang berfungsi sebagai inisialisasi pada stack `s` ke kondisi `nullptr`. Pada baris [12] terdapat `for (int i = 0; i < n; i++) {` yang berfungsi sebagai perulangan for yang berjalan sebanyak n kali untuk membaca nilai input. Selanjutnya pada baris [13] terdapat `std::string simbol;` yang akan digunakan untuk menampung input, baik berupa nilai ataupun simbol. Pada baris [14] terdapat `std::cin >> simbol;` yang digunakan untuk membaca input simbol dan angka, input yang dimasukkan akan disimpan pada variabel `simbol`. Pada baris [16] terdapat pengkondisian `if if (isdigit(simbol[0]) || (simbol.size() > 1 && simbol[0] == '-')) {` yang akan digunakan untuk mengecek apakah input merupakan angka, bukan merupakan simbol operator. Pada baris [17] terdapat fungsi `push(&s, std::stoi(simbol));` yang akan memasukkan angka yang telah diubah dari tipe data string ke tipe data integer dengan menggunakan `std::stoi` ke dalam stack. Pada baris [18] terdapat `} else {` yang akan menjadi alternatif jika input simbol yang dimasukkan adalah simbol operator. Pada baris [20] terdapat `int v2 = peek(&s); pop(&s);` yang akan mengambil nilai teratas dari stack dengan menggunakan fungsi peek untuk disimpan pada variabel `v2`. Setelah nilai teratas dipindahkan, elemen tersebut akan terhapus dari stack agar program dapat mengakses elemen yang berada dibawahnya. Lalu pada baris [21] terdapat `int v1 = peek(&s); pop(&s);` yang memiliki cara kerja yang sama seperti pada baris sebelumnya, namun baris ini mengambil angka yang masuk sebelum `v2` (kedua terakhir). Pemisahan variabel `v1` dan `v2` digunakan agar urutan operasi aritmatika tetap akurat sesuai dengan prinsip LIFO. Pada baris [23] terdapat pengkondisian `if (simbol == "+") push(&s, v1 + v2);` yang akan melakukan operasi penjumlahan antara variabel `v1` dan `v2` dan melakukan push hasilnya ke stack. Kondisi pada baris [23] akan dijalankan jika simbol yang diinput adalah + (simbol penjumlahan). Pada baris [24] terdapat `else if (simbol == "-") push(&s, v1 - v2);` yang akan melakukan proses yang sama seperti baris sebelumnya. Namun kondisi ini melakukan pengurangan antara variabel `v1` dan `v2`. Kondisi ini hanya akan dijalankan jika simbol yang diinput adalah – (simbol pengurangan). Pada baris [25] terdapat kondisi `else if (simbol == "*") push(&s, v1 * v2);` yang akan melakukan operasi perkalian dari nilai `v1` dan `v2` jika simbol yang diinput adalah * (simbol bintang). Hasil dari perkalian `v1` dan `v2` akan dipush ke stack, dan disimpan pada stack `s`. Pada baris [26] terdapat

pengkondisian `else if (simbol == "/") push(&s, v1 / v2);` yang akan melakukan operasi pembagian variabel `v1` dan `v2` jika simbol yang diinput adalah / (garis miring). Pada baris [27] terdapat simbol kurung kurawal yang menjadi penutup dari kondisi `else` pada baris [18]. Pada baris [28] terdapat simbol kurung kurawal yang menjadi penutup dari perulangan `for`. Pada baris [29] terdapat `std::cout << peek(&s) << std::endl;` yang akan menampilkan hasil akhir dari perhitungan. Fungsi `peek` pada baris ini untuk mengambil angka terakhir yang ada pada stack setelah seluruh proses selesai. Pada baris [30] terdapat `return 0;` yang menandakan bahwa program berhasil dijalankan dengan sukses. Terakhir pada baris [31] terdapat simbol tutup kurung yang menutup blok `main` pada program prob1.cpp ini.

SOAL 3: Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;
    return q->rear == &(q->data[MAX - 1]);
}

void enqueue(Queue* q, int value) {
        if   (isFull(q))   throw   std::out_of_range("Queue
Overflow");
```

```
    if (isEmpty(q)) {
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
    } else {
        q->rear++;
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
      if  (isEmpty(q))  throw  std::underflow_error("Queue Underflow");
    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        q->front++;
    }
}

int front(const Queue* q) {
      if  (isEmpty(q))  throw  std::underflow_error("Queue Empty");
    return *(q->front);
}

int back(const Queue* q) {
      if  (isEmpty(q))  throw  std::underflow_error("Queue Empty");
    return *(q->rear);
}
```

B. Pembahasan

1. Penjelasan fungsi: Pada baris [1] terdapat `#include "queue.h"` yang akan digunakan untuk mengimpor definisi dari queue.h dan konstanta MAX agar dapat digunakan dalam program queue.cpp ini. Pada baris [2] terdapat `#include <stdexcept>` yang akan digunakan untuk memanggil standar exception agar `out_of_range` dan `underflow_error` dapat digunakan untuk menangani kondisi error. Pada baris [4] hingga [7] terdapat fungsi init yang digunakan untuk mengatur pointer `front` dan `rear` ke kondisi `nullptr` yang berarti antrean berada pada kondisi kosong saat pertama kali dibuat. Pada baris [9] hingga [11] terdapat fungsi `isEmpty` yang akan mengecek apakah antrean kosong dengan melihat apakah pointer `front` masih bernilai `nullptr`. Pada baris [13] hingga [16] terdapat fungsi `isFull` yang akan memeriksa apakah antrean telah terisi full dengan melakukan perbandingan alamat memori pada pointer `rear` dengan alamat elemen terakhir yang ada pada array `data[MAX - 1]`. Pada baris [18] terdapat fungsi `void enqueue(Queue* q, int value) {` yang akan menambah data ke dalam antrean. Lalu pada baris [19] terdapat exception `if (isFull(q)) throw std::out_of_range("Queue Overflow");` yang akan dijalankan jika antrean terisi penuh. Pada baris [20] hingga [22] terdapat pengkondisian jika data yang dimasukkan adalah elemen pertama dalam array, sehingga `front` dan `rear` akan diarahkan menuju indeks ke-0. Namun, jika `array` sudah memiliki data sebelumnya, baris [24] akan menaikkan alamat pointer `rear` ke posisi berikutnya. Pada baris [26] terdapat `*(q->rear) = value;` yang berfungsi untuk memasukkan nilai `value` pada alamat memori yang telah ditunjuk oleh `rear`. Pada baris [27] terdapat kurung kurawal yang menutup blok dari fungsi `enqueue`. Pada baris [29] terdapat `void dequeue(Queue* q) {` yang akan menghapus data dari antrean. Lalu baris [30] akan menampilkan exception jika antrean kosong. Exception handling pada baris ini akan menghentikan jalannya program jika terdapat queue underflow. Pada baris [31] hingga [33] terdapat pengecekkan apakah data yang dihapus adalah angka terakhir satu-satunya yang ada pada `array`. Jika pengecekkan pada baris sebelumnya terpenuhi, maka `front` dan `rear` akan dikembalikan ke kondisi `nullptr`. Pada baris [34] terdapat pengkondisian `else` yang akan menaikkan alamat pointer `front` ke posisi berikutnya

(maju satu kotak). Proses yang terjadi pada kondisi `else` ini merupakan mekanisme dari prinsip FIFO (First-In-First-Out) yang akan menjadikan elemen pertama masuk akan dilewati pointer saat keluar. Lalu pada baris [36] dan [37] terdapat simbol kurung kurawal yang menutup fungsi else dan dequeue. Pada baris [39] terdapat fungsi front `int front(const Queue* q) {` yang akan melihat nilai elemen terdepan tanpa harus menghapusnya. Pada baris [40] terdapat pengkondisian `if (isEmpty(q)) throw std::underflow_error("Queue Empty");` yang akan mengecek apakah antrean terdeteksi kosong, jika terpenuhi maka baris [40] akan menampilkan error. Sedangkan jika antrean berisi data, program akan mengembalikan nilai yang ditunjuk oleh pointer `front` seperti pada baris [41]. Blok fungsi `front` ditutup oleh simbol kurung kurawal pada baris [42]. Selanjutnya pada baris [44] terdapat fungsi `int back(const Queue* q) {` yang memiliki alur proses seperti sebelumnyam yaitu memeriksa apakah antrean kosong atau memiliki isi, jika antrean kosong maka program akan menampilkan error. Sedangkan pada baris [46] `return *(q->rear);` yang akan mengembalikan nilai yang berada di posisi paling belakang antrean dengan mengambil nilai dari alamat memori yang ditunjuk oleh pointer `rear`. Terakhir, terdapat penutup dari blok fungsi back berupa kurung kurawal.

2. Penjelasan Kebenaran Algoritma: Pada program ini terdapat prinsip FIFO (First-In-First-Out yang akan memastikan bahwa elemen yang pertama kali masuk melalui enqueue menjadi elemen yang pertama kali ditunjuk oleh pointer front untuk dikeluarkan melalui fungsi dequeue. Selain itu program ini menggunakan pointer front dan rear yang memastikan bahwa operasi penambahan dan penghapusan data tidak merusak urutan elemen yang lainnya. Exception kembali digunakan pada program ini agar program tidak melakukan akses memori yang salah.

3. Analisis Kompleksitas (Big-O Notation): Kompleksitas waktu pada fungsi init, isEmpty, isFull, enqueue, dequeue, front, hingga back memiliki kompleksitas waktu yang konstan atau $O(1)$ hal ini dikarenakan setiap fungsi melakukan operasi perbandingan sederhana, sehingga tidak terdapat proses perulangan atau iterasi. Kompleksitas ruang pada setiap fungsi juga bernilai $O(1)$ atau konstan karena fungsi-fungsi tersebut tidak mengalokasikan memori tambahan. Memori yang digunakan pada program ini hanya

digunakan pada beberapa variabel pointer yang ukurannya tetap, terlepas dari kapasitas nilai maksimum MAX yang ditetapkan pada array data.

Soal 4: Si Palui dan Deretan Angka

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buat

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

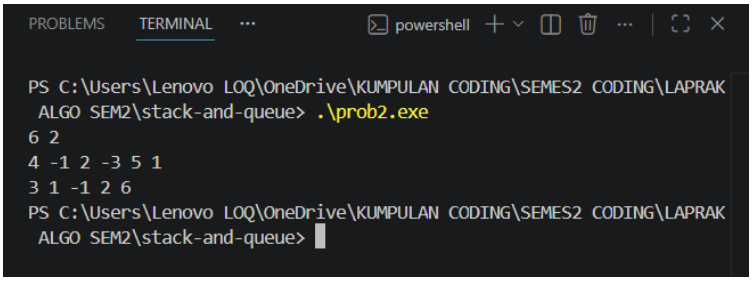
A. Source Code

Tabel 4. Source Code prob2.cpp

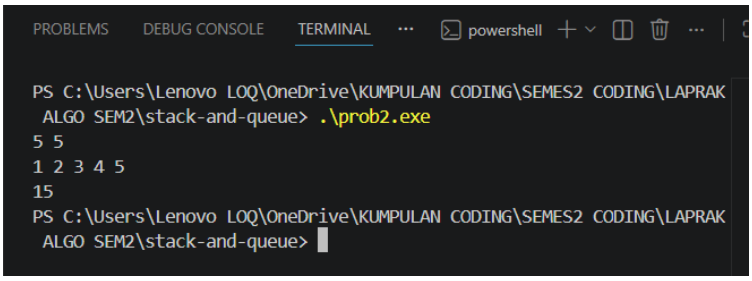
```
#include "queue.h"
#include <iostream>

int main() {
    int n, k;
    if (!(std::cin >> n >> k)) return 0;

    int A[100];
    for (int i = 0; i < n; i++) {
        std::cin >> A[i];
    }

    for (int i = 0; i <= n - k; i++) {
        int sum = 0;

        for (int j = i; j < i + k; j++) {
            sum += A[j];
        }
        std::cout << sum << (i == n - k ? "" : " ");
```

```
20        }
21        std::cout << std::endl;
22        return 0;
23    }
```

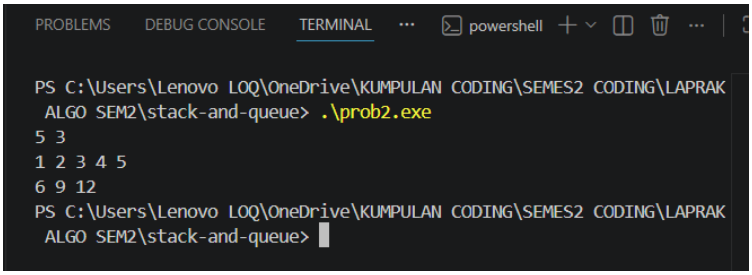
B. Output Program



Gambar 4. Screenshot Hasil Jawaban Soal 4A



Gambar 5. Screenshot Hasil Jawaban Soal 4B



Gambar 6. Screenshot Hasil Jawaban Soal 4C

C. Pembahasan

Pada baris [1] terdapat `#include "queue.h"` yang digunakan untuk mengimpor `queue` agar dapat digunakan pada program ini. Pada baris [2] terdapat `#include <iostream>` yang digunakan untuk mengimpor library `iostream` agar program dapat menggunakan standar input dan output. Pada baris [4] terdapat `int main() {` yang berfungsi sebagai awal pengeksekusian program.Pada baris [5] terdapat deklarasi

variabel `int n, k;`. variabel n akan digunakan untuk jumlah total element, sedangkan variabel `k` akan digunakan untuk menghitung lebar jendela. Variabel `k` akan menjadi penentu batas perulangan pada baris [16], sehingga program tahu kapan akan berhenti untuk melakukan penjumlahan dan mulai mencetak hasil. Lalu pada baris [6] terdapat pengkondisian `if (!(std::cin >> n >> k)) return 0;` yang berfungsi untuk membaca input nilai `n` dan `k`. Baris ini juga akan menghentikan program jika input tidak valid (tidak sesuai dengan tipe data). Pada baris [8] terdapat deklarasi array `int A[100];`. Array ini dapat menyimpan input nilai dengan maksimal 100 elemen. Pada baris [9] hingga [11] terdapat perulangan `for` yang akan digunakan untuk mengambil input dari pengguna dan menyimpannya ke dalam array `A` sebanyak n kali. Pada baris [13] terdapat perulangan `for (int i = 0; i <= n - k; i++) {` yang berfungsi untuk menentukan titik awal dari setiap jendela ynag ingin dihitung. Pada baris ini terdapat batas `n - k` yang akan digunakan agar jendela tetap berada di dalam indeks array. Lalu pada baris [14] terdapat `int sum = 0;` yang digunakan untuk inisialisasi variabel `sum` dan mereset jumlah hitungan setiap kali jendela bergeser ke posisi baru. Lalu pada baris [16] hingga [18] terdapat perulangan bersarang `for (int j = i; j < i + k; j++) {` yang berfungsi untuk melakukan penjumlahan elemen-elemen di dalam jendela berukuran `k`, dan dimulai dari indeks ke-i. Pada baris [17] terdapat `sum += A[j];` yang akan mengumpulkan nilai-nilai yang ada pada jendela saat ini ke dalam variabel sum. Setiap kali perulangan `j` berjalan, variabel `sum` akan bertambah besar hingga seluruh angka dalam satu jendela selesai dihitung. Pada baris [19] terdapat perintah `std::cout << sum << (i == n - k ? "" : " ");` yang akan memastikan spasi hanya berada diantara angka (tidak berada di akhir baris). Lalu pada baris [20] terdapat simbol tutup kurung kurawal yang akan menutup perulangan pertama. Pada baris [21] terdapat `std::cout << std::endl;` yang berfungsi untuk membuat baris baru setelah semua hasil penjumlahan ditampilkan. Pada baris [22] terdapat `return 0;` yang berarti program sukses dijalankan. Pada baris [23] ditutup dengan simbol kurung kurawal yang menutup blok fungsi main.

TAUTAN GIT

https://github.com/laily-gie/Algoritma-dan-Struktur-Data.git